Министерство науки и высшего образования РФ
Федеральное государственное автономное
образовательное учреждение высшего образования
«СИБИРСКИЙ ФЕДЕРАЛЬНЫЙ УНИВЕРСИТЕТ»

Институт космических и информационных технологий
институт

Кафедра программной инженерии
кафедра

ОТЧЕТ О ПРАКТИЧЕСКОЙ РАБОТЕ №4

Процессная аналитика
тема

Преподаватель _____ С. С. Булыкин
подпись, дата — инициалы, фамилия

Студент КИ23-17/2Б, 032320597 _____ А. А. Тисецкая
номер группы, зачётной книжки — подпись, дата — инициалы, фамилия

Красноярск 2026

СОДЕРЖАНИЕ

1 Цель и задачи работы 3
2 Исходные данные и инструменты 3
3 Предварительная обработка данных 4
4 Построение графов бизнес-процесса 5
4.1 Граф непосредственного следования 5
4.2 Модель процесса: сеть Петри и BPMN 8
4.3 Проверка соответствия модели и журнала 11
5 Анализ бизнес-процесса 11
5.1 Узкие места 11
5.2 Циклы и повторная работа 13
5.3 Отклонения 14
5.4 Оптимальные пути 15
5.5 Экземпляры с аномально большой длительностью 15
6 Предложения по оптимизации бизнес-процесса 17
7 Воспроизводимость результатов 17
Заключение 19
Список использованных источников 20
Приложение А 21

1 Цель и задачи работы

Цель работы: знакомство с технологиями Process mining, формирование навыков построения графов бизнес-процессов и их анализа.

Для достижения цели решены следующие задачи:

- выполнить предварительную обработку журнала событий;
- построить граф выбранного бизнес-процесса и модель процесса;
- провести анализ графа на наличие узких мест, циклов, отклонений и оптимальных путей, определить и проанализировать экземпляры процесса, выполнение которых длится намного дольше остальных;
- предложить варианты оптимизации бизнес-процесса.

2 Исходные данные и инструменты

Для анализа выбран открытый журнал событий Sepsis Cases – Event Log из репозитория 4TU.ResearchData [1]. Журнал выгружен из ERP-системы нидерландской больницы и описывает путь пациентов с подозрением на сепсис: от регистрации в приёмном отделении через лабораторные анализы, лечение и госпитализацию до выписки и, возможно, повторного обращения. Экземпляр процесса (case) – один пациент. Журнал содержит 15 214 событий, 1 050 экземпляров и 16 видов действий; у каждого события 32 атрибутов. По описанию набора данных календарные даты рандомизированы, но интервалы между событиями внутри экземпляра сохранены, поэтому временной анализ корректен.

Действия процесса и их смысл приведены в таблице 1.

Таблица 1 – Действия бизнес-процесса

Действие	Содержание
ER Registration	Регистрация пациента в приёмном отделении
ER Triage	Первичная сортировка (оценка срочности)
ER Sepsis Triage	Сортировка по критериям сепсиса
CRP, Leucocytes, LacticAcid	Лабораторные анализы: С-реактивный белок, лейкоциты, лактат
IV Liquid	Внутривенная инфузия жидкости
IV Antibiotics	Внутривенное введение антибиотиков
Admission NC	Госпитализация в обычное отделение (перевод между отделениями)
Admission IC	Госпитализация в отделение интенсивной терапии
Release A–E	Выписка (A – основной тип; B–E – другие исходы)
Return ER	Повторное обращение в приёмное отделение после выписки

Программа написана на языке Python 3. Использованы библиотеки: PM4Py [2] – чтение XES, построение графа непосредственного следования (DFG), алгоритм Inductive Miner, сети Петри, BPMN и проверка соответствия методом воспроизведения маркеров; pandas – предварительная обработка и расчёт показателей; matplotlib – диаграммы; Graphviz – визуализация графов. Текст программы приведён в приложении А.

3 Предварительная обработка данных

Предварительная обработка включала следующие шаги:

- из 32 атрибутов оставлены идентификатор экземпляра (case:concept:name), действие (concept:name), время (time:timestamp) и группа исполнителей (org:group); время приведено к типу datetime в UTC;

- проверены пропуски в ключевых полях – 0; удалены полные дубликаты событий (тот же экземпляр, действие и время) – 7;

- события упорядочены по времени внутри экземпляра устойчивой сортировкой: при равном времени сохранён порядок файла (таких соседних пар 4 440, в основном пакетная регистрация анализов CRP и Leucocytes);

- проверена полнота экземпляров: 55 экземпляров начинаются не с регистрации, 268 не содержат выписки (журнал обрезан на этапе приёмного отделения); для построения модели оставлено 739 полных экземпляров;

- выделен эпизод госпитализации – события от регистрации до первой выписки включительно; повторное обращение Return ER (в среднем через полтора месяца) анализируется отдельно, иначе оно искажает длительность лечения.

Влияние выделения эпизода видно по длительности: по всему журналу медиана 5,3 сут, среднее 28,5 сут, максимум 422,3 сут (в длительность попадает период между выпиской и возвращением); для эпизода госпитализации медиана 5,5 сут, среднее 7,6 сут, 95-й перцентиль 20,2 сут. Сводные показатели приведены в таблице 2.

Таблица 2 – Характеристики журнала событий

Показатель	**Значение**
Событий / экземпляров (исходно)	15 214 / 1 050
Видов действий	16
Полных экземпляров (эпизодов госпитализации)	739
Вариантов (уникальных трасс) во всём журнале	845 (из них единичных 782)
Доля самого частого варианта	3,33 %
Медиана длительности эпизода, сут	5,49
Квартили Q1 / Q3 эпизода, сут	3,76 / 8,97
Максимальная длительность эпизода, сут	114,36

Журнал очень вариативен: 845 вариантов на 1050 экземпляров, самый частый вариант покрывает лишь 3,33 % и обрывается на сортировке. Причина – лабораторные анализы повторяются и регистрируются в разном порядке. Это типичная «спагетти»-структура, поэтому для поиска маршрутов далее используется агрегирование (раздел 5.4).

4 Построение графов бизнес-процесса

4.1 Граф непосредственного следования

Граф непосредственного следования (DFG) построен функцией pm4py.discover_dfg по всему журналу: 115 дуг. Для читаемости на рисунке 1 оставлены дуги с частотой не менее 25 (53 дуг). Число в узле – частота действия, число на дуге – сколько раз одно действие непосредственно следовало за другим.

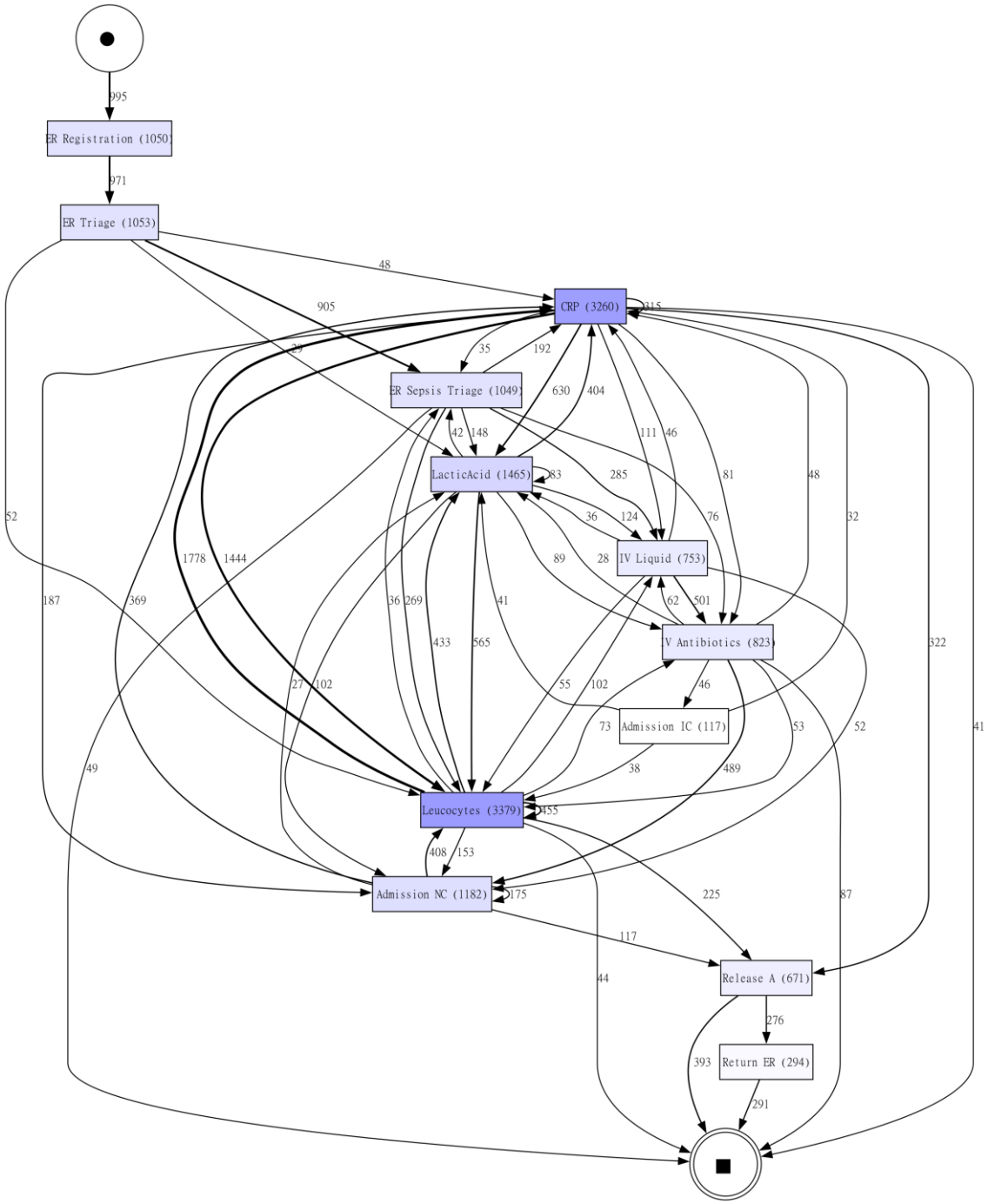


Рисунок 1 – Граф непосредственного следования (частоты переходов)

Основной поток виден по толщине дуг: ER Registration → ER Triage (971) → ER Sepsis Triage (905), далее блок лабораторных анализов с плотными взаимными переходами CRP ⇄ Leucocytes (1 444 и 1 778), лечение IV Liquid → IV Antibiotics (501) → Admission NC (489) и выписка Release A (671). После выписки 276 пациентов вернулись (Release A → Return ER). Петли на узлах Leucocytes, CRP, LacticAcid и Admission NC – повторные анализы и переводы.

Граф производительности (рисунок 2) построен функцией pm4py.discover_performance_dfg: на дугах указана медиана времени перехода.

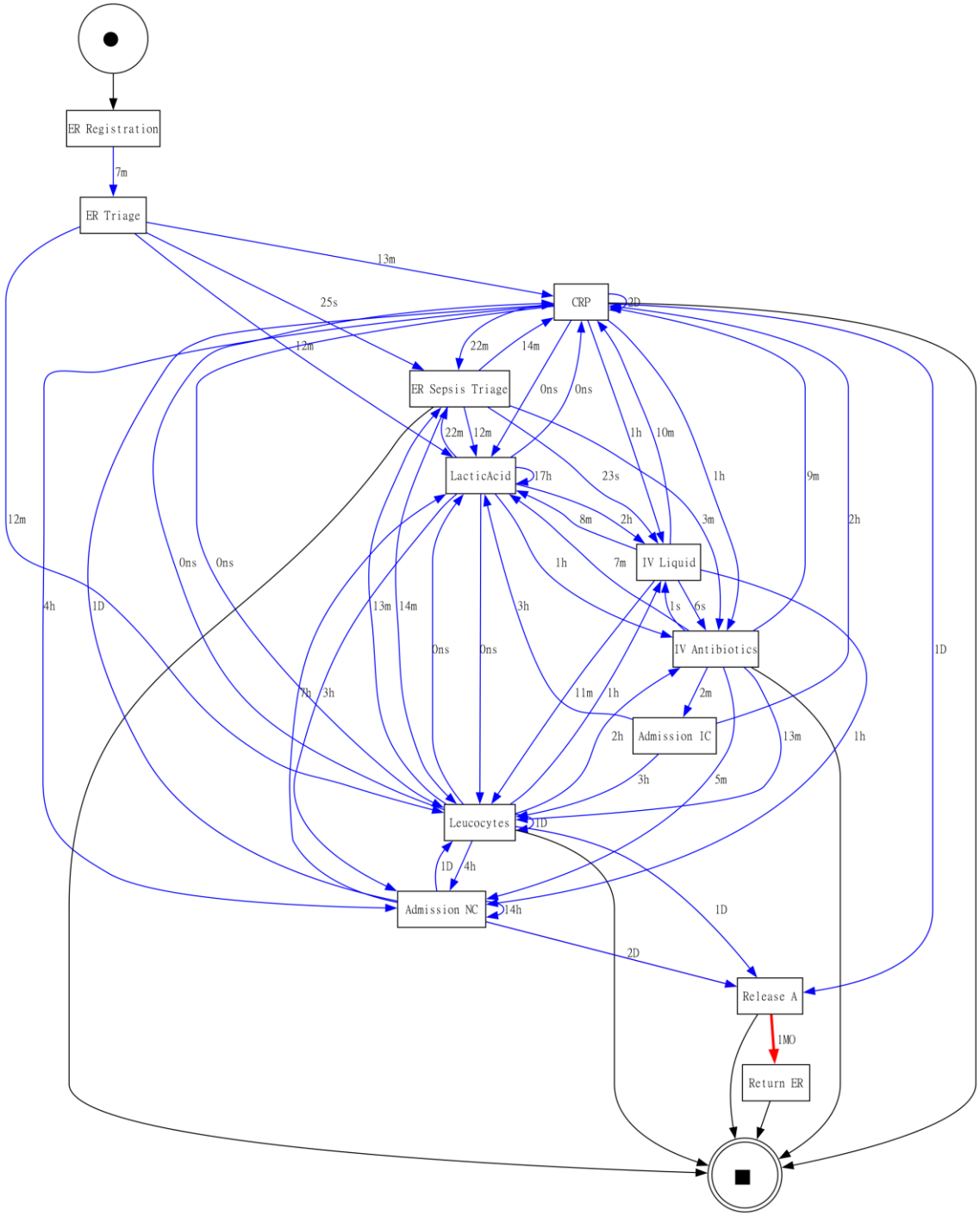


Рисунок 2 – Граф производительности (медиана времени перехода)

Переходы в приёмном отделении занимают минуты, а наибольшее время приходится на дуги внутри стационара: повторные анализы (CRP → CRP – около 2 сут, Leucocytes → Leucocytes – 1 сут), переходы от последнего анализа к

выписке и Admission NC → Release A (около 2,8 сут), а также Release A → Return ER (около 47 сут).

4.2 Модель процесса: сеть Петри и BPMN

Модель процесса построена алгоритмом Inductive Miner с фильтрацией шума (IMf, порог 0,2) по 739 эпизодам госпитализации. Алгоритм рекурсивно разбивает журнал разрезами (последовательность, исключающий выбор, параллельность, цикл) и строит дерево процесса, которое затем преобразуется в сеть Петри (рисунок 3) и BPMN-диаграмму (рисунок 4). Полученное дерево процесса:

->(ER Registration, +(*(Leucocytes, tau), *(Admission NC, tau), *(CRP, tau), ->(ER Sepsis Triage, IV Antibiotics), ->(*(ER Triage, tau), X(tau, *(Admission IC, tau))), *(LacticAcid, tau), X(tau, IV Liquid)), X(Release D, Release E, Release B, Release C, Release A))

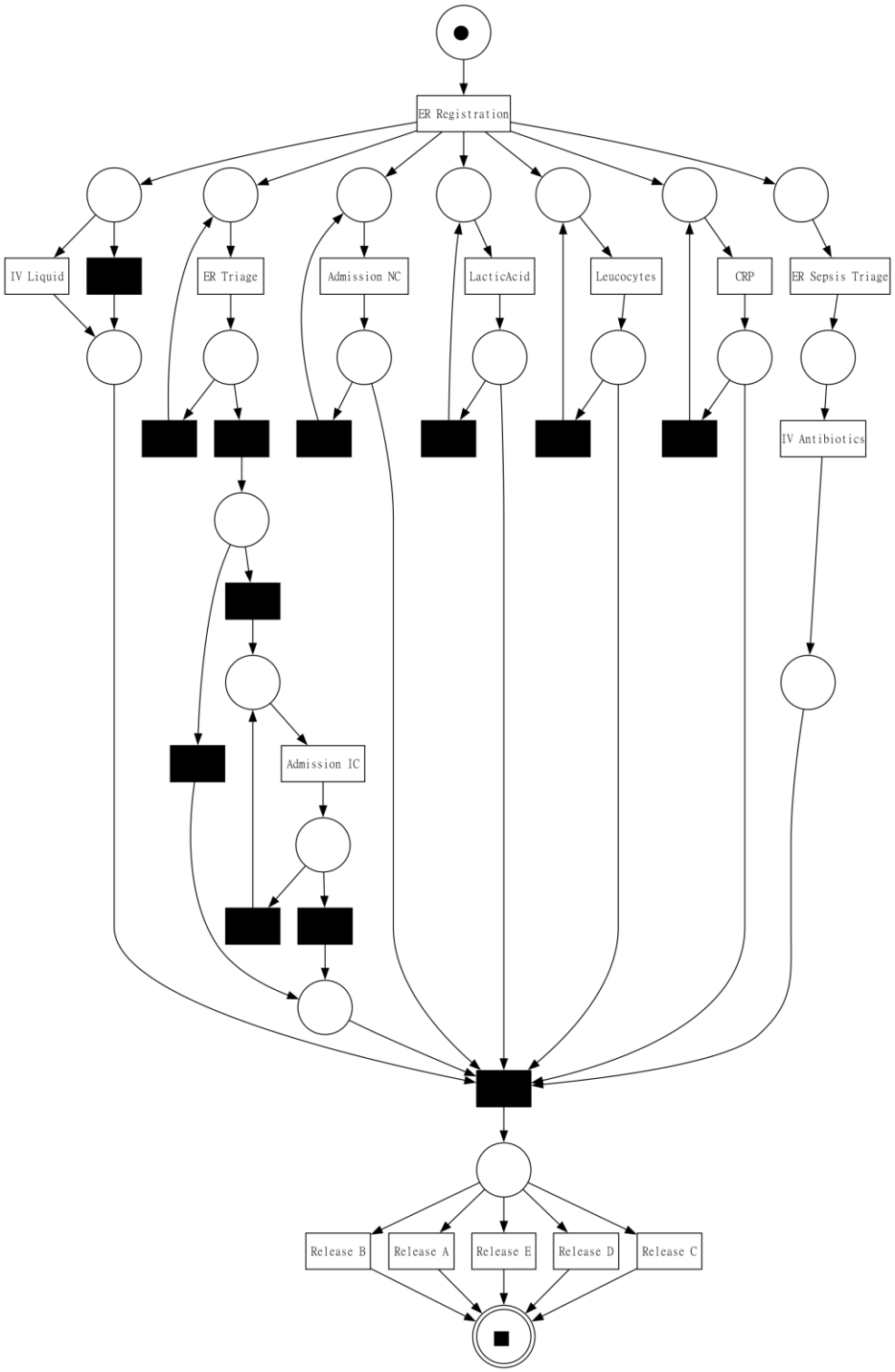


Рисунок 3 – Сеть Петри (Inductive Miner, порог шума 0,2)

Сеть содержит 22 позиций и 27 переходов, из них 12 невидимых (τ, чёрные прямоугольники) – они реализуют пропуски и повторения. Модель читается так: после регистрации параллельно выполняются сортировки, анализы (каждый может повторяться), лечение (ER Sepsis Triage → IV Antibiotics, необязательная

IV Liquid) и госпитализация (с необязательным переводом в Admission IC), после чего следует один из пяти типов выписки.

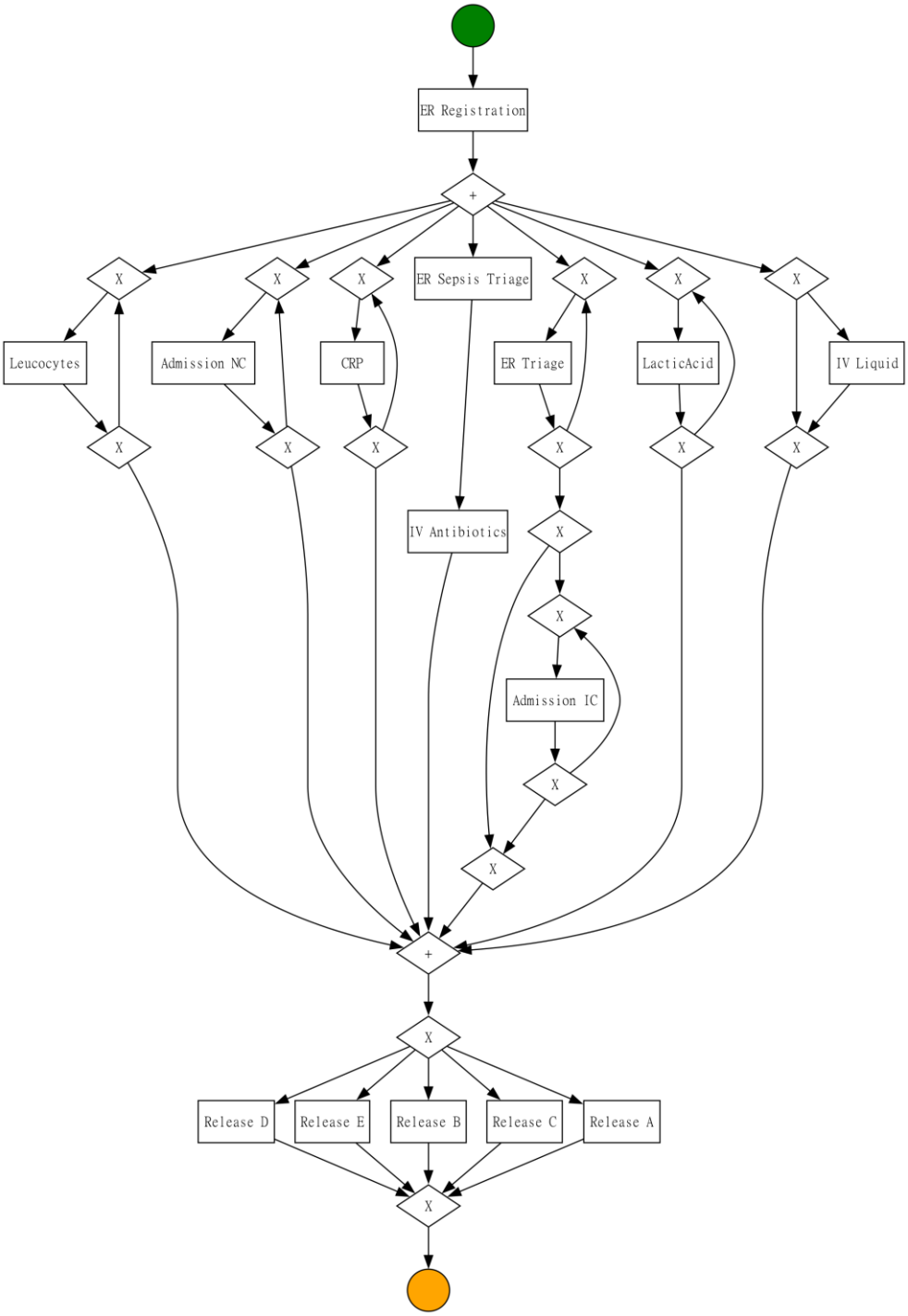


Рисунок 4 – Модель процесса в нотации BPMN

На BPMN-диаграмме ромб «+» – параллельный шлюз (AND), «X» – исключающий шлюз (XOR); обратные дуги между парами XOR-шлюзов задают повторение действия.

4.3 Проверка соответствия модели и журнала

Качество модели оценено методом воспроизведения маркеров (token-based replay): трасса проигрывается на сети Петри, подсчитываются недостающие и оставшиеся маркеры. Пригодность (fitness) показывает, насколько модель воспроизводит журнал, точность (precision) – насколько мало лишнего поведения она допускает. Результаты приведены в таблице 3.

Таблица 3 – Метрики соответствия моделей

Модель	Fitness	Precision	Доля полностью соответствующих трасс
Inductive Miner, порог 0 (без фильтрации)	1,000	0,252	100 %
Inductive Miner, порог 0,2 (итоговая)	0,980	0,371	83,2 %

Модель без фильтрации воспроизводит все трассы, но её точность мала: она разрешает почти любой порядок действий («цветочная» модель). Фильтрация шума повышает precision в 1,5 раза при снижении fitness всего до 0,980 – это компромисс между недообучением и переобучением. 124 эпизодов не соответствуют итоговой модели; это отклонения, рассмотренные в разделе 5.3.

5 Анализ бизнес-процесса

5.1 Узкие места

Узкие места определялись по медиане времени перехода (медиана устойчива к выбросам) и по суммарному времени, которое процесс проводит на дуге. Рассмотрены переходы, встречающиеся не менее 30 раз, в пределах эпизода госпитализации (таблица 4).

Таблица 4 – Переходы с наибольшей медианой времени

Переход	Число	Медиана, ч	90-й перцентиль, ч	Доля общего времени
Admission NC → Release A	111	67,7	125,2	6,4 %
CRP → CRP	292	48,0	92,9	10,6 %
CRP → Release A	309	32,0	99,2	10,8 %
Leucocytes → Release A	212	30,0	99,5	7,2 %
Admission NC → CRP	343	25,9	65,2	8,6 %
Admission NC → Leucocytes	376	24,5	66,3	9,1 %
Leucocytes → Leucocytes	412	24,0	72,0	10,9 %

Выявлены следующие узкие места:

- время пребывания в отделении до выписки: Admission NC → Release A – медиана 2,8 сут; от госпитализации до выписки в целом медиана 5,4 сут, 90-й перцентиль 14,8 сут;

- ожидание повторных анализов: CRP → CRP – 48 ч, Leucocytes → Leucocytes – 24 ч; переходы между анализами в сумме занимают 44,7 % всего времени эпизодов – анализы повторяются по суточному графику, и выписка привязана к их результатам (CRP → Release A, Leucocytes → Release A – 30–32 ч);

- задержка антибиотикотерапии: медиана от ER Sepsis Triage до IV Antibiotics – 1,60 ч, норматив «не позднее 1 ч» выполнен лишь в 40,7 % случаев, в 21,3 % – позднее 3 ч (рисунок 5).

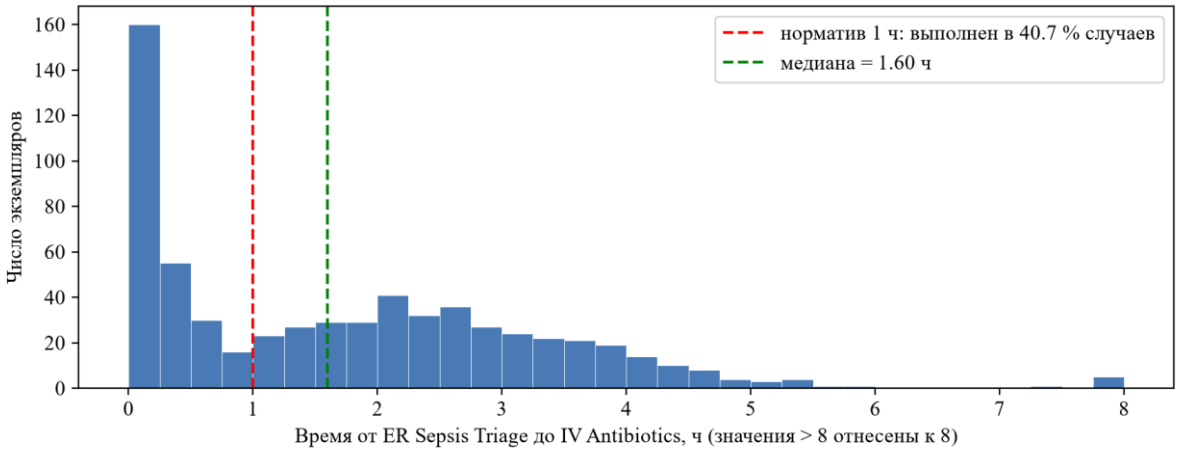


Рисунок 5 – Распределение времени от сортировки по сепсису до введения антибиотиков

Для сравнения: регистрация → сортировка по сепсису занимает в медиане 10,3 мин, регистрация → госпитализация – 3,2 ч, то есть приёмное отделение не является узким местом по длительности, но является им по своевременности лечения.

5.2 Циклы и повторная работа

В графе найдено 40 пар взаимных дуг (двухвершинных циклов A ⇄ B) и 1 029 самопетель (повтор действия подряд). Хотя бы одно действие повторяется в 743 экземплярах (70,8 %). Частота повторов по действиям показана на рисунке 6.

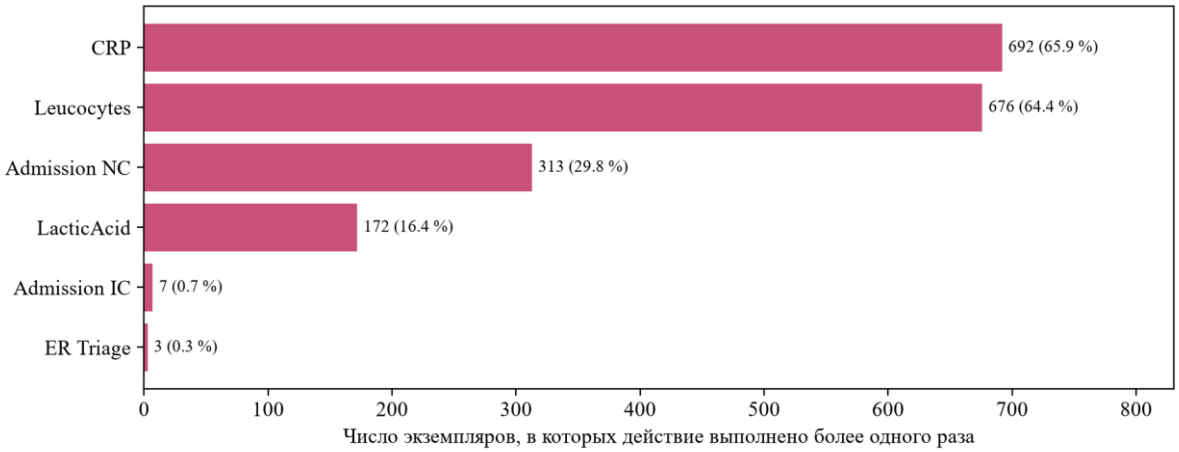


Рисунок 6 – Число экземпляров с повторным выполнением действия

- CRP и Leucocytes повторяются в 65,9 % и 64,4 % экземпляров, до 69 и 74 раз за эпизод; самый плотный цикл графа – CRP ⇄ Leucocytes (1 444 + 1 778 переходов): анализы берутся парой, порядок их регистрации случаен;

- Admission NC повторяется в 29,8 % экземпляров – переводы между отделениями;

- макроцикл «выписка → повторное обращение»: Return ER у 278 из 739 выписанных пациентов (37,6 %), медиана интервала 47,4 сут, 39,2 % возвратов происходят в течение 30 сут.

Число анализов сильно связано с длительностью (коэффициент Спирмена 0,69 для CRP), рисунок 7.

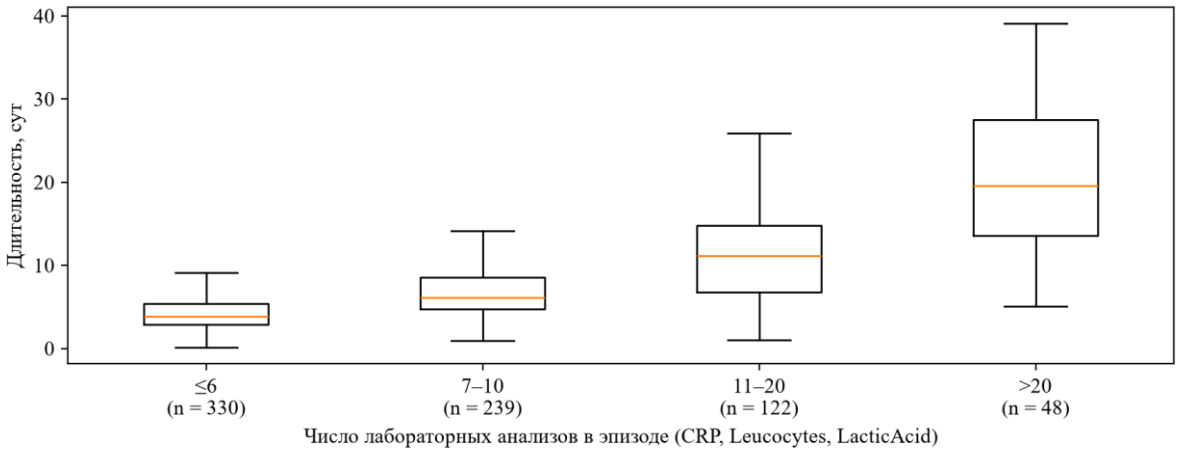


Рисунок 7 – Длительность эпизода в зависимости от числа анализов

5.3 Отклонения

Отклонения выявлены двумя способами: проверкой соответствия модели (раздел 4.3) и проверкой бизнес-правил по трассам. Результаты приведены в таблице 5.

Таблица 5 – Отклонения от правил процесса

Правило / признак	Экземпляров	Доля
Начало не с ER Registration	55	5,2 %
Нет выписки (Release) – незавершённый экземпляр	268	25,5 %
Антибиотики позднее 1 ч после ER Sepsis Triage	481	45,8 %
Госпитализация раньше ER Triage	0	0,0 %
Антибиотики раньше ER Sepsis Triage	0	0,0 %
Выписка без госпитализации	0	0,0 %
Переход в реанимацию (Admission IC)	110	10,5 %
Повторное обращение (Return ER)	294	28,0 %
Не соответствуют модели (token replay)	124	16,8 %

Нарушений порядка (госпитализация раньше сортировки, антибиотики раньше сортировки по сепсису, выписка без госпитализации) не обнаружено – регламентная последовательность соблюдается. Основное отклонение – несвоевременное введение антибиотиков (481 экземпляров). Незавершённые экземпляры – особенность выгрузки, они исключены из построения модели. Не соответствующие модели эпизоды имеют медиану 4,68 сут против 5,65 сут у соответствующих: это в основном редкие переходы (повторная сортировка, нетипичный порядок), а не затяжные случаи.

5.4 Оптимальные пути

Точные варианты почти уникальны, поэтому маршрут пациента агрегирован: лабораторные анализы исключены, подряд идущие повторы объединены. Получено 65 маршрутов вместо 666 вариантов; пять самых частых покрывают 73,6 % эпизодов (таблица 6).

Таблица 6 – Наиболее частые маршруты эпизода госпитализации

Маршрут	Число	Доля	Медиана, сут	Return ER
Reg → Triage → Sepsis → Liquid → AB → NC → Rel A	356	48,2 %	5,21	40 %
Reg → Triage → Sepsis → NC → Rel A	72	9,7 %	4,48	36 %
Reg → Triage → Sepsis → AB → Liquid → NC → Rel A	41	5,5 %	4,64	46 %
Reg → Triage → Sepsis → AB → NC → Rel A	40	5,4 %	4,89	40 %
Reg → Triage → Sepsis → Liquid → AB → IC → NC → Rel A	35	4,7 %	7,13	49 %
Reg → Triage → Sepsis → Liquid → AB → NC → Rel B	19	2,6 %	6,16	0 %

Сокращения: Reg – ER Registration, Triage – ER Triage, Sepsis – ER Sepsis Triage, Liquid – IV Liquid, AB – IV Antibiotics, NC/IC – Admission NC/IC, Rel – Release.

Основной путь (happy path) – Reg → Triage → Sepsis → Liquid → AB → NC → Rel A: 356 эпизодов (48,2 %), медиана 5,21 сут. Самый быстрый из маршрутов с числом эпизодов не менее 10 – Reg → Liquid → Triage → Sepsis → AB → NC → Rel A: медиана 4,29 сут (15 эпизодов), в нём инфузия начата сразу после регистрации. Маршруты с переводом в реанимацию (IC) длиннее: 7,1 и 13,9 сут. Оптимальным следует считать основной путь без перевода в реанимацию и с ранним началом инфузии и антибиотиков; среди точных вариантов самый быстрый с не менее чем 5 экземплярами длится 2,93 сут (Reg → Triage → Sepsis → Leucocytes → CRP → NC → Rel A).

5.5 Экземпляры с аномально большой длительностью

Порог аномальной длительности определён по правилу межквартильного размаха: Q3 + 1,5·IQR = 16,79 сут (рисунок 8). Длиннее порога 55 эпизодов (7,4 %), но на них приходится 25,7 % всего времени пребывания пациентов.

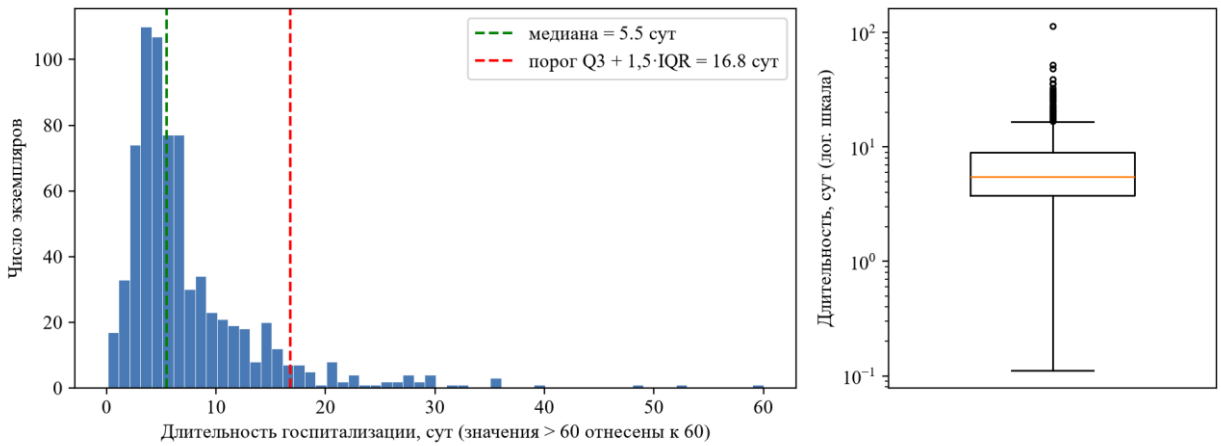


Рисунок 8 – Распределение длительности эпизода госпитализации

Таблица 7 – Сравнение длительных и остальных экземпляров

Показатель	Длительные	Остальные
Медиана длительности, сут	22,2	5,1
Медиана числа событий	29,0	14,0
Медиана числа анализов CRP	9,0	3,0
Медиана числа анализов (CRP, Leucocytes, LacticAcid)	21,0	7,0
Доля с Admission IC, %	40,0	10,8
Медиана числа госпитализаций/переводов	2,0	1,0
Доля с Return ER, %	38,2	37,6
Доля не соответствующих модели, %	27,3	15,9

По таблице 7 длительные экземпляры отличаются прежде всего числом анализов (в 3 раза больше), долей переводов в реанимацию (40 % против 11 %, почти в 4 раза) и числом переводов между отделениями; доля повторных обращений почти не отличается. Время длительных экземпляров в основном тратится на циклы анализов: CRP → Leucocytes, Leucocytes → Leucocytes и CRP → CRP дают около половины их времени.

Пять самых длительных эпизодов: NGA – 114,4 сут, 184 событий, 69 анализов CRP, реанимация; KM – 52,4 сут, 169 событий, 51 анализов CRP, реанимация; LJ – 48,5 сут, 31 событий, 12 анализов CRP; NEA – 39,1 сут, 51 событий, 18 анализов CRP, реанимация; PU – 35,9 сут, 38 событий, 15 анализов CRP. Это тяжёлые случаи с переводами между отделениями и десятками анализов. Если не выделять эпизод, самыми длинными оказались бы экземпляры по 387–422 сут, которые почти целиком состоят из интервала между выпиской и повторным обращением.

6 Предложения по оптимизации бизнес-процесса

По результатам анализа предложены следующие меры:

- внедрить протокол «антибиотик в течение часа» (sepsis bundle): назначение антибиотика одновременно с ER Sepsis Triage, заранее подготовленные наборы препаратов; целевой показатель – доля введений не позднее 1 ч выше текущих 40,7 %;

- стандартизировать лабораторный блок: единая панель CRP + Leucocytes + LacticAcid с одной регистрацией и повтор анализов по клиническим критериям, а не по ежедневному графику; это сократит самопетли и цикл CRP ⇄ Leucocytes; переходы между анализами занимают 44,7 % времени эпизодов;

- планировать выписку заранее: решение о выписке по результатам утренних анализов и ежедневный разбор пациентов, находящихся дольше 17 сут (на 7,4 % эпизодов приходится 25,7 % койко-дней);

- снизить число переводов между отделениями и в реанимацию за счёт раннего выявления ухудшения по лактату: длительные эпизоды почти в 4 раза чаще проходят через Admission IC;

- организовать наблюдение после выписки: 37,6 % пациентов возвращаются, медиана интервала 47,4 сут – контрольный визит или звонок через 2–4 недели после выписки;

- повысить качество данных: фиксировать фактическое время выполнения, а не пакетной регистрации (4 440 пар событий с одинаковым временем), и выгружать экземпляры полностью.

Эффект мер предлагается контролировать теми же показателями после внедрения: временем до антибиотиков, числом анализов на эпизод, медианой длительности госпитализации и долей повторных обращений.

7 Воспроизводимость результатов

Все результаты получены программой analysis.py без ручных правок: при повторном запуске создаются те же таблицы, рисунки и файл summary.json;

генератор случайных чисел зафиксирован. Отчёт формируется скриптом create_report.py из сохранённых результатов. Порядок запуска: установить зависимости (pip install -r requirements.txt) и Graphviz, выполнить python analysis.py, затем python create_report.py. Корректность расчётов проверяется тестами (python -m unittest discover -s tests).

ЗАКЛЮЧЕНИЕ

В работе выполнен процессный анализ пути пациентов с подозрением на сепсис по журналу Sepsis Cases (15 214 событий, 1 050 экземпляров). Проведена предварительная обработка: удалены дубликаты, проверена полнота экземпляров, выделены эпизоды госпитализации. С помощью PM4Py построены граф непосредственного следования (частотный и временной), модель процесса алгоритмом Inductive Miner в виде сети Петри и BPMN; проверка соответствия дала fitness 0,980 и precision 0,371.

Узкие места процесса – ожидание повторных анализов и выписки в стационаре, а также задержка антибиотиков (норматив 1 ч выполнен в 40,7 % случаев). Основные циклы – повторные анализы CRP и Leucocytes (70,8 % экземпляров с повторами) и повторные обращения (37,6 %). Основной путь покрывает 48,2 % эпизодов. Длительные экземпляры (7,4 %) связаны с тяжестью состояния: в 3 раза больше анализов и почти в 4 раза чаще реанимация. Предложены меры по оптимизации: протокол ранней антибиотикотерапии, стандартизация лабораторного блока, планирование выписки и наблюдение после неё. Цель работы достигнута.

СПИСОК ИСПОЛЬЗОВАННЫХ ИСТОЧНИКОВ

1 Mannhardt, F. Sepsis Cases – Event Log // 4TU.ResearchData : репозиторий. – 2016. – DOI 10.4121/uuid:915d2bfb-7e84-49ad-a286-dc35f063a460. – URL: https://data.4tu.nl/articles/dataset/Sepsis_Cases_-_Event_Log/12707639 (дата обращения: 10.11.2026).

2 PM4Py – Process Mining for Python : документация // Fraunhofer FIT : официальный сайт. – URL: https://processintelligence.solutions/pm4py (дата обращения: 10.11.2026).

3 Ван дер Аалст, В. Process Mining: Data Science in Action / В. ван дер Аалст. – 2-е изд. – Berlin : Springer, 2016. – 467 с.

4 Михалев, А. С. Лекция 7. Process mining : конспект лекций по дисциплине «Технологии анализа данных» / А. С. Михалев. – Красноярск : СФУ, 2026. – 52 с.

ПРИЛОЖЕНИЕ А

Листинг программы analysis.py

```
# -*- coding: utf-8 -*-
"""Практическая работа № 4 «Процессная аналитика» (Process mining).

Журнал событий: Sepsis Cases – Event Log (4TU.ResearchData, 2016) –
путь пациентов с подозрением на сепсис через больницу (1 050 случаев).

Этапы:
1. Загрузка и предварительная обработка журнала событий.
2. Построение графов процесса: DFG (частоты и время), сеть Петри и BPMN
   (Inductive Miner), проверка соответствия (token-based replay).
3. Анализ: узкие места, циклы (повторы), отклонения, оптимальные пути,
   экземпляры процесса с аномально большой длительностью.
4. Расчёт показателей для предложений по оптимизации.

Запуск:  python analysis.py            (результаты -> outputs/)
Для рисунков-графов нужен Graphviz (команда dot в PATH); без него
сохраняются исходные описания графов *.gv.
"""
import json
import os
import shutil
import sys
import warnings
from pathlib import Path

import matplotlib
matplotlib.use("Agg")
import matplotlib.pyplot as plt
import numpy as np
import pandas as pd
import pm4py

from pm4py.util import constants as pm_constants

pm_constants.SHOW_PROGRESS_BAR = False
warnings.filterwarnings("ignore")
if hasattr(sys.stdout, "reconfigure"):
    sys.stdout.reconfigure(encoding="utf-8")

ROOT = Path(__file__).resolve().parent
LOG_PATH = ROOT / "data" / "Sepsis Cases - Event Log.xes.gz"
OUT = ROOT / "outputs"
FIG = OUT / "figures"
TAB = OUT / "tables"
for d in (FIG, TAB):
    d.mkdir(parents=True, exist_ok=True)

CASE, ACT, TS = "case:concept:name", "concept:name", "time:timestamp"
RELEASES = ["Release A", "Release B", "Release C", "Release D", "Release E"]
ADMISSIONS = ["Admission NC", "Admission IC"]
LABS = ["CRP", "Leucocytes", "LacticAcid"]
NOISE = 0.2          # порог шума Inductive Miner (IMf)
MIN_EDGE = 25        # порог частоты дуги на рисунке DFG
SEED = 42
```

```
np.random.seed(SEED)

plt.rcParams.update({"font.family": "Times New Roman", "font.size": 11})
S = {}                # сводка для отчёта


def save_table(df, name, index=False):
    df.to_csv(TAB / name, index=index, encoding="utf-8-sig")


def hours(td):
    return td.dt.total_seconds() / 3600


# ============================================================ 1. Загрузка
raw = pm4py.read_xes(str(LOG_PATH), show_progress_bar=False)
S["raw_events"] = int(len(raw))
S["raw_cases"] = int(raw[CASE].nunique())
S["raw_columns"] = int(raw.shape[1])

# ============================================== 2. Предварительная обработка
log = raw[[CASE, ACT, TS, "org:group"]].copy()
log[TS] = pd.to_datetime(log[TS], utc=True)
S["missing_key_values"] = int(log[[CASE, ACT, TS]].isna().sum().sum())
S["missing_org_group"] = int(log["org:group"].isna().sum())
log["org:group"] = log["org:group"].fillna("?")
dups = log.duplicated(subset=[CASE, ACT, TS])
S["duplicates"] = int(dups.sum())
log = log[~dups]
# устойчивая сортировка: при равном времени сохраняется порядок файла
log["_pos"] = np.arange(len(log))
log = log.sort_values([CASE, TS, "_pos"],
kind="stable").drop(columns="_pos").reset_index(drop=True)
same_ts = log.groupby(CASE)[TS].diff().eq(pd.Timedelta(0))
S["equal_timestamp_pairs"] = int(same_ts.sum())

g = log.groupby(CASE)
first_act = g[ACT].first()
last_act = g[ACT].last()
has_release = g[ACT].apply(lambda s: s.isin(RELEASES).any())
S["start_activities"] = first_act.value_counts().to_dict()
S["end_activities"] = last_act.value_counts().to_dict()
S["cases_not_start_registration"] = int((first_act != "ER Registration").sum())
S["cases_without_release"] = int((~has_release).sum())

# Полные экземпляры: начинаются с регистрации в приёмном отделении и
# содержат выписку (Release A-E). Остальные – незавершённые/усечённые.
complete_ids = first_act[(first_act == "ER Registration") & has_release].index
S["complete_cases"] = int(len(complete_ids))
S["incomplete_cases"] = S["raw_cases"] - S["complete_cases"]
clog = log[log[CASE].isin(complete_ids)].reset_index(drop=True)
S["complete_events"] = int(len(clog))
S["activities"] = int(log[ACT].nunique())
S["period"] = [str(log[TS].min().date()), str(log[TS].max().date())]
log.to_csv(OUT / "clean_log.csv", index=False, encoding="utf-8-sig")

# Эпизод госпитализации: от первого события до первой выписки.
# Return ER (повторное обращение) – отдельный эпизод, его не смешиваем
# с длительностью лечения.
```

```
is_rel = clog[ACT].isin(RELEASES).astype(int)
# число выписок строго до текущего события: оставляем события до первой выписки
включительно
rel_before = is_rel.groupby(clog[CASE]).cumsum() - is_rel
stay = clog[rel_before == 0].reset_index(drop=True)
S["post_release_events_removed"] = int(len(clog) - len(stay) - (clog[ACT] ==
"Return ER").sum())

# ======================================== 3. Характеристики экземпляров
def case_table(df):
    gg = df.groupby(CASE)
    t = pd.DataFrame({
        "start": gg[TS].min(), "end": gg[TS].max(), "n_events": gg.size(),
        "variant": gg[ACT].apply(lambda s: " → ".join(s)),
        "last": gg[ACT].last(),
    })
    t["days"] = (t["end"] - t["start"]).dt.total_seconds() / 86400
    return t



cases_all = case_table(log)
d_all = cases_all["days"]
S["duration_all_days"] = {k: round(float(v), 2) for k, v in d_all.describe(
    percentiles=[.25, .5, .75, .9, .95]).items()}

cases = case_table(stay)                                    # эпизоды
госпитализации
gs = stay.groupby(CASE)
cases["n_crp"] = gs[ACT].apply(lambda s: (s == "CRP").sum())
cases["n_leuco"] = gs[ACT].apply(lambda s: (s == "Leucocytes").sum())
cases["n_lab"] = gs[ACT].apply(lambda s: s.isin(LABS).sum())
cases["n_admission"] = gs[ACT].apply(lambda s: s.isin(ADMISSIONS).sum())
cases["icu"] = gs[ACT].apply(lambda s: (s == "Admission IC").any())
cases["release"] = gs[ACT].apply(lambda s: s[s.isin(RELEASES)].iloc[0])
ret = clog.groupby(CASE)[ACT].apply(lambda s: (s == "Return ER").any())
cases["return_er"] = ret.reindex(cases.index).fillna(False)
d = cases["days"]
S["duration_stay_days"] = {k: round(float(v), 2) for k, v in d.describe(
    percentiles=[.25, .5, .75, .9, .95]).items()}

# ================================================== 4. Варианты
def variants_table(c):
    v = c.groupby("variant").agg(cases=("days", "size"), median_days=("days",
"median"),
                                 mean_days=("days", "mean")).sort_values(
        ["cases", "median_days"], ascending=[False, True])
    v["share_%"] = v["cases"] / len(c) * 100
    v["cum_%"] = v["share_%"].cumsum()
    return v



var_all = variants_table(cases_all)
var = variants_table(cases)
S["variants_all"] = int(len(var_all))
S["variants_all_single"] = int((var_all["cases"] == 1).sum())
S["variants_stay"] = int(len(var))
S["variants_stay_single"] = int((var["cases"] == 1).sum())
S["top_variant_share_all"] = round(float(var_all["share_%"].iloc[0]), 2)
save_table(var_all.head(10).round(2).reset_index(), "variants_top10_all.csv")
save_table(var.head(10).round(2).reset_index(), "variants_top10_stay.csv")
```

```
# ================================================ 5. Переходы (DFG)
def transitions(df):
    t = df.copy()
    t["next"] = t.groupby(CASE)[ACT].shift(-1)
    t["next_ts"] = t.groupby(CASE)[TS].shift(-1)
    t = t.dropna(subset=["next"])
    t["h"] = hours(t["next_ts"] - t[TS])
    e = t.groupby([ACT, "next"]).agg(count=("h", "size"), cases=(CASE,
"nunique"),
                                     median_h=("h", "median"), mean_h=("h",
"mean"),
                                     p90_h=("h", lambda x: x.quantile(.9)),
                                     total_h=("h", "sum")).reset_index()
    return t, e.rename(columns={ACT: "from", "next": "to"})



tr_all, edges_all = transitions(log)
tr, edges = transitions(stay)
S["distinct_transitions_all"] = int(len(edges_all))
save_table(edges_all.sort_values("count", ascending=False).round(2),
"dfg_edges_all.csv")

# ============================================ 6. Графы процесса (pm4py)
HAS_DOT = shutil.which("dot") is not None



def save_gviz(gviz, name):
    """Сохранить описание графа (.gv) и, если есть Graphviz, PNG."""
    (FIG / f"{name}.gv").write_text(gviz.source, encoding="utf-8")
    if HAS_DOT:
        gviz.format = "png"
        gviz.render(filename=name, directory=str(FIG), cleanup=True)



from pm4py.visualization.dfg import visualizer as dfg_vis
from pm4py.visualization.petri_net import visualizer as pn_vis
from pm4py.visualization.bpmn import visualizer as bpmn_vis


dfg, sa, ea = pm4py.discover_dfg(log)
perf, _, _ = pm4py.discover_performance_dfg(log)
dfg_f = {k: v for k, v in dfg.items() if v >= MIN_EDGE}
S["dfg_edges_total"] = len(dfg)
S["dfg_edges_shown"] = len(dfg_f)
shown = {a for e in dfg_f for a in e}
act_count = {k: int(v) for k, v in log[ACT].value_counts().items() if k in
shown}
sa = {k: v for k, v in sa.items() if k in shown and v >= MIN_EDGE}
ea = {k: v for k, v in ea.items() if k in shown and v >= MIN_EDGE}
params = {"start_activities": sa, "end_activities": ea, "format": "png",
"bgcolor": "white",
          "rankdir": "TB"}
save_gviz(dfg_vis.apply(dfg_f, activities_count=act_count,
variant=dfg_vis.Variants.FREQUENCY,
                        parameters=params), "dfg_frequency")
perf_f = {k: v for k, v in perf.items() if k in dfg_f}
save_gviz(dfg_vis.apply(perf_f, activities_count=act_count,
variant=dfg_vis.Variants.PERFORMANCE,
                        parameters={**params, "aggregation_measure": "median"}),
"dfg_performance")
```

```
# Модель процесса: Inductive Miner (infrequent) по полным экземплярам
# (эпизоды госпитализации); дерево процесса строится один раз и
# преобразуется в сеть Петри и BPMN, затем проверка соответствия эпизодов.
tree = pm4py.discover_process_tree_inductive(stay, noise_threshold=NOISE)
net, im, fm = pm4py.convert_to_petri_net(tree)
bpmn = pm4py.convert_to_bpmn(tree)
save_gviz(pn_vis.apply(net, im, fm, parameters={"format": "png", "bgcolor":
"white",
                                                "set_rankdir": "TB"}),
"petri_net")
save_gviz(bpmn_vis.apply(bpmn, parameters={"format": "png", "bgcolor": "white",
"rankdir": "TB"}), "bpmn")
S["process_tree"] = str(tree)
S["petri_places"] = len(net.places)
S["petri_transitions"] = len(net.transitions)
S["petri_silent"] = sum(t.label is None for t in net.transitions)

fit = pm4py.fitness_token_based_replay(stay, net, im, fm)
prec = pm4py.precision_token_based_replay(stay, net, im, fm)
S["fitness"] = round(fit["log_fitness"], 4)
S["fitting_traces_%"] = round(fit["percentage_of_fitting_traces"], 2)
S["precision"] = round(prec, 4)
S["f1"] = round(2 * S["fitness"] * S["precision"] / (S["fitness"] +
S["precision"]), 4)
# для сравнения: модель без фильтрации шума
net0, im0, fm0 = pm4py.discover_petri_net_inductive(stay, noise_threshold=0.0)
S["fitness_im0"] = round(pm4py.fitness_token_based_replay(stay, net0, im0,
fm0)["log_fitness"], 4)
S["precision_im0"] = round(pm4py.precision_token_based_replay(stay, net0, im0,
fm0), 4)
diag = pm4py.conformance_diagnostics_token_based_replay(stay, net, im, fm,

return_diagnostics_dataframe=True)
nonfit = set(diag.loc[~diag["is_fit"], "case_id"])
cases["fit"] = ~cases.index.isin(nonfit)
S["nonfit_cases"] = len(nonfit)
S["nonfit_median_days"] = round(float(cases.loc[~cases.fit, "days"].median()),
2)
S["fit_median_days"] = round(float(cases.loc[cases.fit, "days"].median()), 2)

# ================================================ 7. Узкие места
b = edges[edges["count"] >= 30].copy()
b["share_time_%"] = b["total_h"] / edges["total_h"].sum() * 100
bott_med = b.sort_values("median_h", ascending=False).head(8)
bott_tot = b.sort_values("total_h", ascending=False).head(8)
save_table(bott_med.round(2), "bottlenecks_by_median.csv")
save_table(bott_tot.round(2), "bottlenecks_by_total.csv")
lab_edges = edges["from"].isin(LABS) & edges["to"].isin(LABS)
S["lab_block_time_share_%"] = round(float(edges.loc[lab_edges, "total_h"].sum()
                                          / edges["total_h"].sum() * 100), 1)

# время до антибиотиков (клинический регламент: не позднее 1 ч после
# выявления сепсиса)
def first_time(df, a):
    return df[df[ACT] == a].groupby(CASE)[TS].min()



t_tri = first_time(stay, "ER Sepsis Triage")
t_reg = first_time(stay, "ER Registration")
t_ab = first_time(stay, "IV Antibiotics")
```

```
t_adm = stay[stay[ACT].isin(ADMISSIONS)].groupby(CASE)[TS].min()
t_rel = stay[stay[ACT].isin(RELEASES)].groupby(CASE)[TS].min()
ab_h = hours(t_ab - t_tri).dropna()
S["antibiotics_cases"] = int(len(ab_h))
S["antibiotics_median_h"] = round(float(ab_h.median()), 2)
S["antibiotics_within_1h_%"] = round(float((ab_h <= 1).mean() * 100), 1)
S["antibiotics_over_3h_%"] = round(float((ab_h > 3).mean() * 100), 1)
reg_tri = hours(t_tri - t_reg).dropna()
S["registration_to_triage_median_min"] = round(float(reg_tri.median() * 60), 1)
los = (hours(t_rel - t_adm) / 24).dropna()
S["admission_to_release_median_days"] = round(float(los.median()), 2)
S["admission_to_release_p90_days"] = round(float(los.quantile(.9)), 2)
er_phase = hours(t_adm - t_reg).dropna()
S["registration_to_admission_median_h"] = round(float(er_phase.median()), 2)

# ================================================== 8. Циклы
self_loops = edges_all[edges_all["from"] ==
edges_all["to"]].sort_values("count", ascending=False)
save_table(self_loops[["from", "count", "cases", "median_h"]].round(2),
"self_loops.csv")
S["self_loop_events"] = int(self_loops["count"].sum())
cnt = log.groupby([CASE, ACT]).size().unstack(fill_value=0)
rework = pd.DataFrame({
    "cases_with_repeat": (cnt > 1).sum(),
    "share_%": (cnt > 1).mean() * 100,
    "extra_events": (cnt - 1).clip(lower=0).sum(),
    "max_per_case": cnt.max(),
}).sort_values("cases_with_repeat", ascending=False)
rework = rework[rework["cases_with_repeat"] > 0]
save_table(rework.round(2).reset_index(), "rework.csv")
S["cases_with_any_repeat"] = int((cnt > 1).any(axis=1).sum())
S["cases_with_any_repeat_%"] = round(S["cases_with_any_repeat"] / S["raw_cases"]
* 100, 1)
# двухвершинные циклы A ⇄ B
pairs = edges_all.set_index(["from", "to"])["count"].to_dict()
two = [(a, b2, pairs[(a, b2)], pairs[(b2, a)]) for (a, b2) in pairs
       if a < b2 and (b2, a) in pairs]
two = pd.DataFrame(two, columns=["A", "B", "A→B", "B→A"])
two["sum"] = two["A→B"] + two["B→A"]
save_table(two.sort_values("sum", ascending=False), "two_node_cycles.csv")
S["two_node_cycles"] = int(len(two))
# макроцикл: повторное обращение после выписки
S["return_er_cases"] = int(ret.sum())
S["return_er_%"] = round(float(ret.mean() * 100), 1)
rr = tr_all[(tr_all[ACT].isin(RELEASES)) & (tr_all["next"] == "Return ER")]["h"]
/ 24
S["release_to_return_median_days"] = round(float(rr.median()), 1)
S["return_within_30d_%_of_returns"] = round(float((rr <= 30).mean() * 100), 1)
# число анализов CRP в эпизоде и его связь с длительностью
S["crp_per_case_median"] = float(cases["n_crp"].median())
S["corr_days_crp"] = round(float(cases[["days",
"n_crp"]].corr(method="spearman").iloc[0, 1]), 3)

# ================================================ 9. Отклонения
def before(s, a, b2):
    """оба события есть в трассе и первое a стоит позже первого b2
    (то есть b2 произошло раньше a)"""
    s = list(s)
    return a in s and b2 in s and s.index(a) > s.index(b2)



rules = {
```

```
    "Начало не с ER Registration": (first_act != "ER Registration"),
    "Нет выписки (Release) – незавершённый экземпляр": ~has_release,
    "Антибиотики позднее 1 ч после ER Sepsis Triage": (hours(
        first_time(log, "IV Antibiotics") - first_time(log, "ER Sepsis Triage"))
> 1
    ).reindex(first_act.index).fillna(False),
    "Госпитализация раньше ER Triage": log.groupby(CASE)[ACT].apply(
        lambda s: before(s, "ER Triage", "Admission NC")),
    "Антибиотики раньше ER Sepsis Triage": log.groupby(CASE)[ACT].apply(
        lambda s: before(s, "ER Sepsis Triage", "IV Antibiotics")),
    "Выписка без госпитализации": log.groupby(CASE)[ACT].apply(
        lambda s: s.isin(RELEASES).any() and not s.isin(ADMISSIONS).any()),
    "Переход в реанимацию (Admission IC)": log.groupby(CASE)[ACT].apply(
        lambda s: (s == "Admission IC").any()),
    "Повторное обращение (Return ER)": log.groupby(CASE)[ACT].apply(
        lambda s: (s == "Return ER").any()),
}
dev = pd.DataFrame({"cases": {k: int(v.sum()) for k, v in rules.items()}})
dev["share_%"] = (dev["cases"] / S["raw_cases"] * 100).round(1)
dev.loc["Не соответствуют модели (token replay)", ["cases", "share_%"]] = [
    S["nonfit_cases"], round(S["nonfit_cases"] / S["complete_cases"] * 100, 1)]
save_table(dev.reset_index().rename(columns={"index": "rule"}),
"deviations.csv")
S["deviations"] = dev["cases"].astype(int).to_dict()

# ================================================ 10. Оптимальные пути
happy = var.index[0]
S["happy_path"] = happy
S["happy_path_cases"] = int(var["cases"].iloc[0])
S["happy_path_median_days"] = round(float(var["median_days"].iloc[0]), 2)
fast = var[var["cases"] >= 5].sort_values("median_days").head(5)
save_table(fast.round(2).reset_index(), "fastest_variants.csv")
S["fastest_variant"] = fast.index[0]
S["fastest_variant_cases"] = int(fast["cases"].iloc[0])
S["fastest_variant_median_days"] = round(float(fast["median_days"].iloc[0]), 2)

# Точные варианты почти уникальны (лабораторные анализы повторяются в
# разном порядке), поэтому маршрут пациента агрегируется: лабораторные
# анализы исключаются, подряд идущие повторы схлопываются.
def route(s):
    out = []
    for a in s:
        if a in LABS:
            continue
        if not out or out[-1] != a:
            out.append(a)
    return " → ".join(out)



cases["route"] = gs[ACT].apply(route)
routes = cases.groupby("route").agg(cases=("days", "size"), median_days=("days",
"median"),
                                    icu=("icu", "mean"), return_er=("return_er",
"mean"))
routes["share_%"] = routes["cases"] / len(cases) * 100
routes = routes.sort_values("cases", ascending=False)
save_table(routes.head(10).round(3).reset_index(), "routes_top10.csv")
S["routes"] = int(len(routes))
S["routes_top5_cum_%"] = round(float(routes["share_%"].head(5).sum()), 1)
S["happy_route"] = routes.index[0]
S["happy_route_cases"] = int(routes["cases"].iloc[0])
S["happy_route_share_%"] = round(float(routes["share_%"].iloc[0]), 1)
```

```
S["happy_route_median_days"] = round(float(routes["median_days"].iloc[0]), 2)
rfast = routes[routes["cases"] >= 10].sort_values("median_days")
S["fastest_route"] = rfast.index[0]
S["fastest_route_cases"] = int(rfast["cases"].iloc[0])
S["fastest_route_median_days"] = round(float(rfast["median_days"].iloc[0]), 2)
save_table(rfast.head(5).round(3).reset_index(), "routes_fastest.csv")
# сравнение групп по числу анализов (CRP+Leucocytes+LacticAcid)
cases["lab_group"] = pd.cut(cases["n_lab"], [0, 6, 10, 20, 1000],
                            labels=["≤6", "7–10", "11–20", ">20"],
include_lowest=True)
lab_tab = cases.groupby("lab_group", observed=False).agg(
    cases=("days", "size"), median_days=("days", "median"),
    icu_share=("icu", "mean"), return_share=("return_er", "mean"))
save_table(lab_tab.round(3).reset_index(), "lab_groups.csv")

# ======================================== 11. Длительные экземпляры
q1, q3 = d.quantile(.25), d.quantile(.75)
thr = q3 + 1.5 * (q3 - q1)
S["long_threshold_days"] = round(float(thr), 2)
cases["long"] = cases["days"] > thr
L, N = cases[cases.long], cases[~cases.long]
S["long_cases"] = int(len(L))
S["long_share_%"] = round(len(L) / len(cases) * 100, 1)
S["long_time_share_%"] = round(float(L["days"].sum() / cases["days"].sum() *
100), 1)
cmp_ = pd.DataFrame({
    "Длительные": [L.days.median(), L.n_events.median(), L.n_crp.median(),
L.n_lab.median(),
                   L.icu.mean() * 100, L.n_admission.median(),
L.return_er.mean() * 100,
                   (~L.fit).mean() * 100],
    "Остальные": [N.days.median(), N.n_events.median(), N.n_crp.median(),
N.n_lab.median(),
                  N.icu.mean() * 100, N.n_admission.median(), N.return_er.mean()
* 100,
                  (~N.fit).mean() * 100],
}, index=["Медиана длительности, сут", "Медиана числа событий", "Медиана числа
анализов CRP",
          "Медиана числа анализов (CRP, Leucocytes, LacticAcid)", "Доля с
Admission IC, %",
          "Медиана числа госпитализаций/переводов", "Доля с Return ER, %",
          "Доля не соответствующих модели, %"])
save_table(cmp_.round(2).reset_index(), "long_vs_normal.csv")
trL = tr[tr[CASE].isin(L.index)]
wl = trL.groupby([ACT, "next"])["h"].agg(["size", "median",
"sum"]).reset_index()
wl["share_%"] = wl["sum"] / wl["sum"].sum() * 100
save_table(wl.sort_values("sum", ascending=False).head(6).round(2),
"long_where_time.csv")
top_long = cases.sort_values("days", ascending=False).head(5)[
    ["days", "n_events", "n_crp", "n_admission", "icu", "release", "return_er"]]
save_table(top_long.round(2).reset_index(), "top5_long_cases.csv")
# для сравнения: длительность всего наблюдения (с Return ER)
S["top_long_all_cases"] = cases_all.sort_values("days",
ascending=False).head(5)[
    ["days", "last"]].round(1).reset_index().to_dict("records")

# ================================================ 12. Рисунки
def savefig(fig, name):
    fig.tight_layout()
    fig.savefig(FIG / name, dpi=200)
    plt.close(fig)
```

```
fig, (a1, a2) = plt.subplots(1, 2, figsize=(10, 3.8),
gridspec_kw={"width_ratios": [2.2, 1]})
a1.hist(d.clip(upper=60), bins=60, color="#4a7ab5", edgecolor="white", lw=.3)
a1.axvline(d.median(), color="green", ls="--", label=f"медиана =
{d.median():.1f} сут")
a1.axvline(thr, color="red", ls="--", label=f"порог Q3 + 1,5·IQR = {thr:.1f}
сут")
a1.set_xlabel("Длительность госпитализации, сут (значения > 60 отнесены к 60)")
a1.set_ylabel("Число экземпляров")
a1.legend()
a2.boxplot(d, vert=True, widths=.5, flierprops={"markersize": 3})
a2.set_yscale("log")
a2.set_ylabel("Длительность, сут (лог. шкала)")
a2.set_xticks([])
savefig(fig, "duration_distribution.png")

fig, ax = plt.subplots(figsize=(9, 3.8))
top = var_all.head(10).iloc[::-1]
ax.barh([f"Вариант {i}" for i in range(10, 0, -1)], top["cases"],
color="#4a7ab5")
for i, (c, sh) in enumerate(zip(top["cases"], top["share_%"])):
    ax.text(c + .4, i, f"{c} ({sh:.1f} %)", va="center", fontsize=9)
ax.set_xlim(0, top["cases"].max() * 1.25)
ax.set_xlabel("Число экземпляров процесса")
savefig(fig, "variants_top10.png")

fig, ax = plt.subplots(figsize=(9, 3.6))
rw = rework.head(8).iloc[::-1]
ax.barh(rw.index, rw["cases_with_repeat"], color="#c8507a")
for i, (c, sh) in enumerate(zip(rw["cases_with_repeat"], rw["share_%"])):
    ax.text(c + 5, i, f"{c} ({sh:.1f} %)", va="center", fontsize=9)
ax.set_xlim(0, rw["cases_with_repeat"].max() * 1.2)
ax.set_xlabel("Число экземпляров, в которых действие выполнено более одного
раза")
savefig(fig, "rework.png")

fig, ax = plt.subplots(figsize=(9, 3.6))
ax.hist(ab_h.clip(upper=8), bins=np.arange(0, 8.25, .25), color="#4a7ab5",
edgecolor="white", lw=.3)
ax.axvline(1, color="red", ls="--", label=f"норматив 1 ч: выполнен в
{S['antibiotics_within_1h_%']} % случаев")
ax.axvline(ab_h.median(), color="green", ls="--", label=f"медиана =
{ab_h.median():.2f} ч")
ax.set_xlabel("Время от ER Sepsis Triage до IV Antibiotics, ч (значения > 8
отнесены к 8)")
ax.set_ylabel("Число экземпляров")
ax.legend()
savefig(fig, "antibiotics_delay.png")

fig, ax = plt.subplots(figsize=(9, 3.6))
data = [cases.loc[cases.lab_group == k, "days"] for k in lab_tab.index]
ax.boxplot(data, tick_labels=[f"{k}\n(n = {n})" for k, n in zip(lab_tab.index,
lab_tab["cases"])],
           showfliers=False)
ax.set_xlabel("Число лабораторных анализов в эпизоде (CRP, Leucocytes,
LacticAcid)")
ax.set_ylabel("Длительность, сут")
savefig(fig, "labs_vs_duration.png")

# ================================================ 13. Сводка
```

```
cases.reset_index().to_csv(OUT / "cases.csv", index=False, encoding="utf-8-sig")
with open(OUT / "summary.json", "w", encoding="utf-8") as f:
    json.dump(S, f, ensure_ascii=False, indent=2, default=str)
print(json.dumps(S, ensure_ascii=False, indent=2, default=str))
print("Graphviz:", "найден, PNG графов сохранены" if HAS_DOT else
      "не найден – сохранены только описания графов *.gv")
```